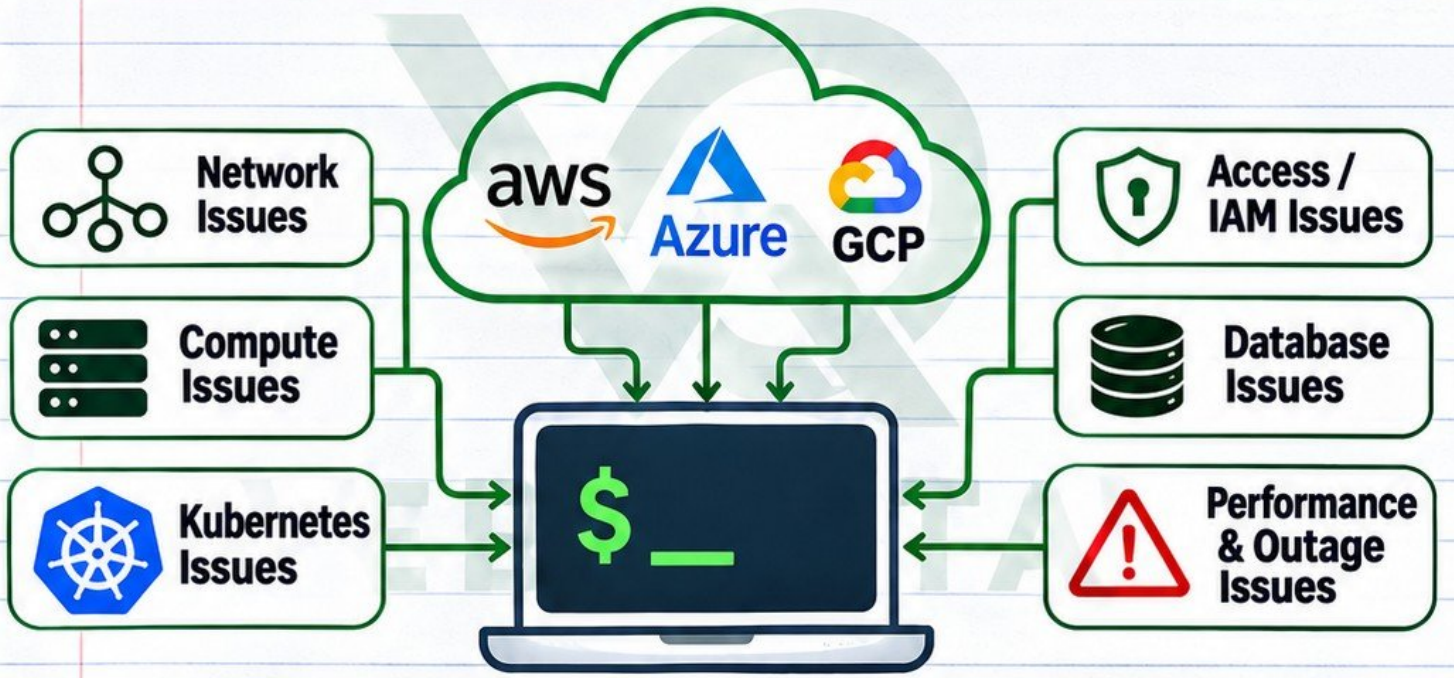
VERIQTA

Cloud Troubleshooting

Commands Handbook

20 Production Incident Investigations

Diagnose • Investigate • Fix • Verify



Real-World Cloud Scenarios for DevOps, SRE, and Platform Engineers

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

The Server Is Running, but the Application Is Unreachable

Trace socket listeners, interfaces, routes, firewalls, and service dependencies.

Scenario

The server is up and healthy (SSH works), but the application (e.g., web, API, or database) cannot be reached by users or other services.

Investigation Flow

1. Confirm the service is running and listening.
2. Check network interfaces and IP binding.
3. Validate routing to the application port.
4. Check firewall rules (OS, cloud, and security groups).
5. Verify upstream or dependent services.



Key Commands and Expected Output

Command	Expected Output (Example)
sudo systemctl status app	● active (running)
sudo ss -tulnp | grep 8080	LISTEN 0.0.0.0:8080 users:("app",pid=1234)
ip addr show	inet 10.0.2.10/24 ...
ip route	default via 10.0.2.1 dev eth0
sudo ufw status	Status: active 8080/tcp ALLOW
sudo iptables -S | grep 8080	-A INPUT -p tcp --dport 8080 -j ACCEPT
curl -I http://localhost:8080	HTTP/1.1 200 OK
curl -I http://10.0.2.10:8080	HTTP/1.1 200 OK

Interpret the Evidence

- Service is running and listening on the expected port.
- The correct IP address is bound to the service.
- Routing points to the right network/gateway.
- No firewall or security group is blocking access.
- Dependencies (DB, API, cache) are reachable and healthy.

Common Root Causes

- Service bound to 127.0.0.1 (localhost only).
- Port blocked by OS firewall or cloud security group.
- Incorrect or missing route to the target network.
- NACL or upstream firewall denying traffic.
- A dependent service (DB/API/cache) is unavailable.

Fix and Verify

1. Bind the application to the correct interface (e.g., 0.0.0.0 or server IP).
2. Open the required port in the OS firewall and cloud security group.
3. Update any incorrect route or network rule.
4. Restore or start the dependent service if unavailable.
5. Verify end-to-end access with curl or a browser.

Prove It's Resolved

curl -I http://<server-ip>:8080	HTTP/1.1 200 OK
ss -tulnp | grep 8080	LISTEN ... :8080
sudo systemctl startus app	active (running)

VERIQTA

The Network Works, but Requests Randomly Time Out

Diagnose TCP retransmissions, packet loss, connection tracking, and exhausted sockets.

Scenario

Network connectivity (ping, DNS, basic reachability) works, but application requests intermittently time out (e.g., web, API, or database).

Investigation Flow

1. Check for packet loss and high latency.
2. Inspect TCP retransmissions.
3. Analyze connection tracking and NAT state.
4. Verify socket usage and limits on the server.
5. Correlate findings and identify the bottleneck.



Key Commands and Expected Output

Command	Expected Output (Example)
ping -c 10 <target-ip>	10 packets, 0–5% loss, avg 45 ms
mtr -rw <target-ip>	Shows hop latency and packet loss
ss -s	TCP: established 3,842, orphaned 0, time-wait 12,560
ss -tan state time-wait | wc -l	12560 (high TIME_WAIT)
sudo conntrack -S	entries 973412, max 1048576, use 92%
netstat -s | grep -i retrans	TcpRetransSegs: 18432
cat /proc/sys/net/ipv4/ip_local_port_range	32768 60999

Interpret the Evidence

- Packet loss >1–2% can cause timeouts and retries.
- High TCP retransmissions indicate network congestion or drops.
- High conntrack usage (>80%) can drop new connections.
- Many sockets in TIME_WAIT may exhaust available ports.
- Intermittent latency spikes often correlate with timeouts.

Common Root Causes

- Packet loss or network congestion.
- NAT/connection tracking table nearly full.
- Too many open or TIME_WAIT sockets.
- Low ephemeral port range (port exhaustion).
- Overloaded or slow downstream service.

Fix and Verify

1. Reduce connection churn (use keep-alive/connection pooling).
2. Increase conntrack and socket limits (if required).
3. Tune TCP and port range: `-w net.ipv4. ip_local_port_range="10240 65535"`
4. Fix network congestion or packet loss at the affected hop.
5. Re-test and monitor for stability.

Verification

mtr -rw <target-ip>	0% loss, stable latency
ss -s	Normal socket counts
sudo conntrack -S	Use < 70%
curl -w "%{time_total}\n" http://app	Consistent response time (e.g., 0.12s)

VERIQTA

DNS Resolves Correctly, but the Application Connects to the Wrong Backend

Investigate split-horizon DNS, resolver caches, TTLs, and stale endpoints.

Scenario

The DNS lookup returns a valid IP address, but the application connects to an unexpected or outdated backend (e.g., old IP, wrong environment, or region).

Investigation Flow

1. Confirm the DNS resolution result from multiple locations.
2. Check for split-horizon DNS (internal vs external).
3. Inspect DNS cache and TTL values.
4. Verify the backend endpoint and traffic routing.
5. Identify stale records and update DNS if needed.



Key Commands and Expected Output

Command	Expected Output (Example)
dig +short app.example.com	10.0.1.10
dig app.example.com	TTL: 3600, ANSWER: 10.0.1.10
nslookup app.example.com	Address: 10.0.1.10
ipconfig /flushdns (Windows)	Succesfully flushed DNS cache
sudo systemd-resolve --flush-caches (Linux)	(No output, cache cleared)
dig @8.8.8.8 app.example.com	10.0.2.20 (public/expected IP)

Interpret the Evidence

- Different IPs internally and externally indicate split-horizon DNS.
- A high TTL (e.g., 3600s) can keep clients on a stale IP.
- The resolver cache may still hold the old record.
- If the IP points to an old or wrong backend, DNS propagation or caching is the cause.

Common Root Causes

- Split-horizon DNS (internal vs external).
- Stale DNS records due to long TTL.
- DNS cache not yet refreshed on clients or resolvers.
- Wrong CNAME or environment DNS record.

Fix and Verify

1. Update the DNS record to the correct backend IP.
2. Reduce the TTL (e.g., 300s) for faster propagation.
3. Clear DNS caches on clients, servers, and resolvers.
4. Validate resolution from multiple networks (internal and external).
5. Confirm the application connects to the correct backend.

Verification

dig +short app.example.com	10.0.2.20
curl -I http://app.example.com	HTTP/1.1 200 OK
nslookup app.example.com	Address: 10.0.2.20
dig app.example.com | grep TTL	TTL: 300

VERIQTA

The Private Subnet That Suddenly Lost Internet Access

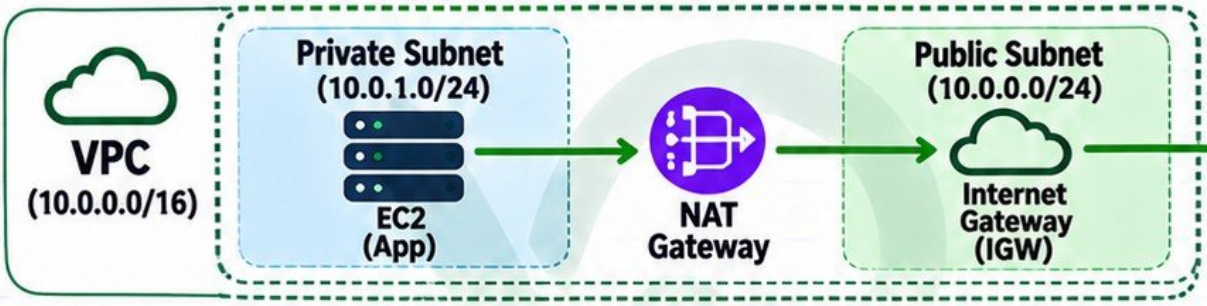
Investigate NAT gateways, route tables, security groups, NACLs, and ephemeral ports.

Scenario

Instances in a private subnet that previously accessed the internet (e.g., updates, APIs) suddenly cannot reach external resources.

Investigation Flow

1. Verify outbound connectivity (e.g., curl, ping).
2. Check the route table and default route (0.0.0.0/0).
3. Confirm NAT gateway health and status.
4. Review Security Group and NACL outbound rules.
5. Check for ephemeral port exhaustion or limits.



Key Commands and Expected Output

Command	Expected Output (Example)
curl -I https://www.google.com	(Hangs or timeout)
ip route show	default via 10.0.0.1 dev eth0
aws ec2 describe-route-tables --route-table-ids rtb-123	0.0.0.0/0 → nat-0abc123
aws ec2 describe-nat-gateways --nat-gateway-ids nat-0abc123	State: available
sudo ss -s	TCP: inuse 48000, timewait 9200
cat /proc/sys/net/ipv4/ip_local_port_range	32768 60999

Interpret the Evidence

- No response to curl indicates no internet connectivity.
- Route table must have 0.0.0.0/0 → NAT gateway.
- NAT gateway should be in "available" state.
- Security Group must allow outbound traffic (e.g., 0.0.0.0/0).
- NACL must allow outbound and return traffic (ephemeral ports, e.g., 1024–65535).

Common Root Causes

- Missing or incorrect route to the NAT gateway.
- NAT gateway deleted, stopped, or unhealthy.
- Security Group or NACL blocking outbound traffic.
- Exhausted ephemeral ports (too many connections).
- NAT gateway hitting bandwidth or connection limits.

Fix and Verify

1. Add or correct the default route (0.0.0.0/0 → NAT gateway).
2. Restore or replace the NAT gateway (if not available).
3. Allow outbound traffic in the Security Group (e.g., 0.0.0.0/0, required ports).
4. Update the NACL to allow outbound and ephemeral return ports.
5. Test again with curl and monitor NAT gateway metrics (connections, bandwidth).

Verification

curl -I https://www.google.com	HTTP/1.1 200 OK
aws ec2 describe-nat-gateways	State: available
aws ec2 describe-route-tables	0.0.0.0/0 → nat-...
ss -s	Connections stable

The Load Balancer Is Healthy, but Users Receive 502 and 504 Errors

Correlate backend connection failures, TLS handshakes, target health, and upstream timeouts.

Scenario

The load balancer reports healthy, but users receive intermittent 502 (Bad Gateway) and 504 (Gateway Timeout) errors.

Investigation Flow

1. Check target health and backend connectivity.
2. Analyze TLS handshake and certificate issues.
3. Review load balancer and target logs/metrics.
4. Verify upstream timeouts (LB and application).
5. Correlate findings to identify the root cause.



Key Commands and Expected Output

Command	Expected Output (Example)
aws elbv2 describe-target-health --target-group-arn <tg-arn>	healthy / unhealthy (503, timeout, 5xx)
curl -I http://app.example.com	HTTP/1.1 502 or 504
openssl s_client -connect backend:443 -servername app.example.com	TLS handshake details (OK or error)
aws elbv2 describe-load-balancers --names <lb-name>	State: active, Health: healthy
tail -n 50 /var/log/nginx/error.log	upstream timed out (110) or 502/504

Interpret the Evidence

- HTTP 502: backend connection failure (e.g., refused, reset, or invalid response).
- HTTP 504: the backend did not respond within the configured timeout.
- TLS handshake failures can cause 502 errors (e.g., certificate, SNI, or cipher issues).
- A "healthy" load balancer only means the LB itself is up, not that backends are responding correctly or fast enough.

Common Root Causes

- Backend service down or overloaded.
- TLS certificate or handshake misconfiguration.
- Load balancer timeout lower than application time.
- Network or security group/ACL blocking traffic.
- Upstream dependency (API/DB) is slow or failing.

Fix and Verify

1. Restore backend service health or increase capacity.
2. Fix TLS configuration (certificate, SNI, cipher).
3. Align timeouts (LB, application, and upstream).
4. Ensure security groups/ACLs allow required traffic.
5. Retest and monitor error rate and target health.

Verification

curl -I http://app.example.com	HTTP/1.1 200 OK
aws elbv2 describe-target-health ...	healthy
openssl s_client -connect backend:443 ...	Verify: Handshake OK
Check LB metrics (HTTPCode_502/504)	Drops to ~0

The EC2 Instance Has Low CPU but Is Still Unresponsive

Diagnose I/O wait, memory pressure, disk queues, and process scheduling.

Scenario

An EC2 instance shows low CPU utilization (e.g., 5–10%) but the application is slow or unresponsive.

Investigation Flow

1. Check CPU, I/O wait, and load averages.
2. Verify memory usage and pressure.
3. Inspect disk I/O and queue depth.
4. Analyze running processes and scheduling.
5. Identify the bottleneck and take corrective action.



Key Commands and Expected Output

Command	Expected Output (Example)
top / htop	%CPU: 5.0, %wa: 42.0 (high I/O wait)
vmstat 1 5	wa: 40 43 41 (I/O wait high)
iostat -xz 1 5	%util: 96.4, await: 320ms (disk busy/slow)
free -h	Mem: 7.6G used / 8.0G (no free memory)
sar -B 1 5	pgpgin/pgpgout high (memory pressure)
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head	Top processes (e.g., java, node, nginx) using high memory

Interpret the Evidence

- High I/O wait (%wa) indicates disk bottleneck, not CPU.
- High memory usage or swap activity indicates memory pressure.
- High disk queue (await, %util) means slow or overloaded storage.
- A few processes may be consuming memory or blocking I/O.

Common Root Causes

- Disk I/O bottleneck (slow or saturated EBS).
- Memory pressure (high usage, swap thrashing).
- High disk queue depth (backlog of read/write).
- A blocking or runaway process (e.g., DB, Java).
- I/O heavy workload (logs, backups, large queries).

Fix and Verify

1. Increase IOPS or move to a higher EBS volume type (e.g., gp3/io2).
2. Free memory or add more RAM (stop memory-hungry processes).
3. Optimize the offending process (e.g., DB query, log rotation).
4. Re-test and confirm lower I/O wait and stable response time.

Verification

top (after fix)	%wa < 5.0
iostat -xz 1 3	%util < 70, await < 50ms
free -h	Enough free memory (e.g., ≥1G)
curl -I http://<app>	HTTP/1.1 200 OK (responsive)

The Container Is Running, but Its Application Is Dead

Inspect process state, namespaces, health checks, and container networking.

Scenario

The container is in a Running state, but the application inside is not responding (e.g., no process, crashed, or unhealthy).

Investigation Flow

1. Verify container status and health.
2. Check if the application process is running.
3. Inspect logs and recent container events.
4. Review namespaces and resource limits.
5. Test container networking and service reachability.



Key Commands and Expected Output

Command	Expected Output (Example)
docker ps	CONTAINER ID STATUS abc123 Up 10m (healthy)
docker exec -it <id> ps -ef	(Shows app process) or (no app process found)
docker logs --tail 50 <id>	Error: connection refused or application crash message
docker inspect <id> | grep -i health	"Status": "unhealthy" (failed checks)
docker exec -it <id> ss -tulnp	LISTEN 0 128 0.0.0.0:8080 (If app is running)

Interpret the Evidence

- Container Running only means the container exists, not that the app is healthy.
- No application process = the app crashed, exited, or never started.
- Unhealthy health check indicates the app is not responding as expected.
- Namespace or resource limits (CPU/Mem) can prevent the app from starting.
- Network test helps confirm if the app port is open and reachable inside the container.

Common Root Causes

- The application process crashed on startup.
- Wrong or missing start command (CMD/ENTRYPOINT).
- Failed or misconfigured health check.
- Resource limits (CPU or memory) too low.
- The app is running but not listening on the expected port.

Fix and Verify

1. Start or fix the application process (correct CMD).
2. Review and update the health check (e.g., HTTP or TCP).
3. Adjust resource limits (CPU/memory) if needed.
4. Confirm the app is listening on the expected port.
5. Re-test connectivity from inside and outside the container.

Verification

docker exec -it <id> ps -ef	App process is running
curl -I http://localhost:8080	HTTP/1.1 200 OK
docker inspect <id> | grep -i health	"Status": "healthy"
docker logs --tail 20 <id>	No errors, app started
docker ps	Up (healthy)

Kubernetes Pods Are Ready, but the Service Has No Working Endpoints

Trace Services, EndpointSlices, readiness probes, and traffic routing.

Scenario

Pods are in Ready state, but the Service shows no endpoints, so traffic cannot reach the application.

Investigation Flow

1. Check the Service and its selector.
2. Inspect EndpointSlices and endpoints.
3. Verify pod labels match the Service selector.
4. Review readiness probe status and events.
5. Test traffic routing from inside the cluster.



Key Commands and Expected Output

Command	Expected Output (Example)
kubectl get pods	web-6d7f8c6b9-abc 1/1 Ready
kubectl get svc web	ClusterIP 10.96.21.5 <none>
kubectl get endpoints web	Endpoints: <none>
kubectl get endpointslice -l kubernetes.io/service-name=web	ADDRESS CONDITIONS <empty>
kubectl describe svc web	Selector: app=web Endpoints: <none>
kubectl get pod -l app=web --show-labels	app=web

Interpret the Evidence

- Service has no endpoints even though pods are Ready.
- No EndpointSlice endpoints means the pods are not selected by the Service.
- Common causes: label mismatch, readiness probe failing, or incorrect selector.
- Pod may be Ready (container running) but not Ready for traffic if the probe is misconfigured.

Common Root Causes

- Service selector does not match pod labels.
- Readiness probe is failing or takes too long.
- Wrong port (containerPort vs Service targetPort).
- EndpointSlice not created (e.g., controller issue).
- NetworkPolicy blocking service traffic.

Fix and Verify

1. Align the Service selector with the pod labels.
2. Fix the readiness probe (path, port, or timing).
3. Ensure the Service targetPort matches the containerPort.
4. Recreate or update the Service if needed.
5. Verify endpoints and test traffic.

Verification

kubectl get endpoints web	Shows pod IPs (e.g., 10.244.1.7)
kubectl get endpointslice -l kubernetes.io/service-name=web	Endpoints listed (Ready: true)
kubectl run test --rm -it --image=busybox -- wget -qO- http://web	Returns app response (e.g., 200 OK)

The Kubernetes Node Is Ready, but Workloads Keep Failing

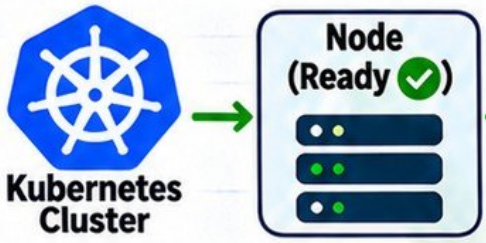
Investigate kubelet, container runtime, disk pressure, CNI, and kernel logs.

Scenario

The Kubernetes node is in Ready state, but pods and workloads keep failing (e.g., CrashLoopBackOff, ImagePullError, or create errors).

Investigation Flow

1. Check kubelet health and logs.
2. Verify container runtime (containerd or Docker).
3. Check node disk pressure and resource availability.
4. Inspect CNI (network plugin) for errors.
5. Review kernel logs (dmesg) for system issues.



Key Components to Check

Kubelet | Container Runtime (containerd) | Disk Pressure | CNI (Calico/Flannel) | Kernel Logs (dmesg)

Key Commands and Expected Output

Command	Expected Output (Example)
kubectl get nodes	NAME STATUS ROLES node-1 Ready <none>
systemctl status kubelet	● active (running)
journalctl -u kubelet -n 50	kubelet logs (scheduling / container errors)
crictl ps -a	List of containers (containerd)
df -h	Filesystem Use% /dev/xvda1 92%
dmesg | tail -n 20	Kernel messages (e.g., OOM, I/O)
kubectl describe node node-1	Conditions: DiskPressure: True (or False)

Interpret the Evidence

- Kubelet errors indicate pod startup or runtime issues.
- Container runtime (containerd/Docker) may be down, misconfigured, or unstable.
- High disk usage can trigger DiskPressure and block new pods.
- CNI failures can prevent pod networking and cause pod creation errors.
- Kernel logs may reveal low memory (OOM), I/O errors, or other system-level problems.

Common Root Causes

- Kubelet not healthy or misconfigured.
- Container runtime (containerd/Docker) failure.
- Disk pressure or full disk (logs, images, data).
- CNI plugin errors (e.g., Calico, Flannel, Cilium).
- Kernel issues (OOM, I/O errors, or module failures).

Fix and Verify

1. Restart or fix kubelet (systemctl restart kubelet).
2. Fix container runtime (e.g., restart containerd).
3. Free up disk space (clean logs, unused images).
4. Fix or restart the CNI plugin (e.g., calico, flannel).
5. Recreate the workload and confirm it runs.

Verification

kubectl get pods -o wide	Pods in Running state
systemctl status kubelet	● active (running)
crictl info	Runtime: containerd (OK)
df -h	Use% < 80% (no pressure)
kubectl get nodes	Ready (no DiskPressure)

Autoscaling Increased Capacity and Made the Outage Worse

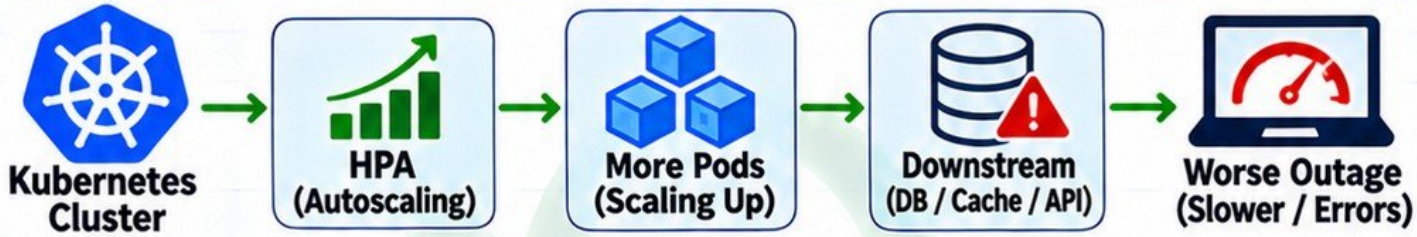
Diagnose HPA behavior, pod startup, resource limits, and downstream saturation.

Scenario

Autoscaling (HPA) added more pods to handle high load, but the application became slower and less available.

Investigation Flow

1. Check HPA configuration and scaling events.
2. Analyze pod startup time and readiness.
3. Review resource requests and limits.
4. Check downstream systems (DB, cache, APIs) for saturation.
5. Correlate scaling, traffic, and error rate in metrics/logs.



Key Commands and Expected Output

Command	Expected Output (Example)
kubectl get hpa	NAME TARGETS MIN MAX web-hpa 85%/60% 2 20
kubectl describe hpa web-hpa	Events: Scaled up to 10 pods (High CPU: 85%)
kubectl get pods -l app=web	10/10 Running (AGE: 1m ...)
kubectl top pods -l app=web	CPU: 750m MEM: 1.2Gi (spike, higher than normal)
kubectl describe pod <pod>	Readiness: succeeded Startup time: 90s

Interpret the Evidence

- HPA scaled up because metrics (CPU/Memory) exceeded the target threshold.
- New pods take time to start (image pull, initialization, readiness).
- Resource limits may cause CPU throttling or OOM, making new pods slow.
- Downstream systems (e.g., database, cache) can become saturated by the sudden spike.
- Overall, scaling increased load and amplified the bottleneck, worsening the outage.

Common Root Causes

- HPA configured with a too-low target (overreacts).
- Slow pod startup (large image or heavy initialization).
- Incorrect or tight resource requests/limits.
- Downstream saturation (DB, cache, external API).
- No rate limiting or backpressure to protect dependencies.

Fix and Verify

1. Tune HPA (e.g., higher target, stabilizationWindow).
2. Optimize pod startup (smaller image, pre-pull, readiness).
3. Set appropriate resource requests and limits.
4. Protect downstream services (connection limits, queue, rate limiting).
5. Test under load and validate stable scaling and performance.

Verification

kubectl get hpa	TARGETS: 45% / 60% (stable)
kubectl get pods -l app=web	Stable replicas (e.g., 4/4 Running)
kubectl top pods	CPU/MEM within expected range
App health check (curl)	HTTP/1.1 200 OK (responsive)
Downstream (DB/cache) metrics	No saturation (normal latency)

The IAM Policy Allows Access, but AWS Still Returns AccessDenied

Investigate SCPs, permission boundaries, session policies, and resource policies.

Scenario

The IAM policy appears to allow the action, but AWS still returns AccessDenied (e.g., when calling an API or accessing a resource).

Investigation Flow

1. Confirm the identity, policy, and action.
2. Check for an explicit deny in SCPs.
3. Review any permission boundaries.
4. Inspect session policies (STS, role assumption).
5. Check the resource policy for the target resource.



Key Commands and Expected Output

Command	Expected Output (Example)
aws sts get-caller-identity	Account: 123456789012 Arn: arn:aws:sts::...:role/Dev
aws iam simulate-principal-policy --policy-source-arn <arn> --action-names s3:ListBucket	EvalDecision: allowed (or explicitDeny)
aws organizations describe-effective-policy --policy-type SERVICE_CONTROL_POLICY	SCP: Deny s3:* (If an org-level deny exists)
aws iam get-role --role-name <role>	PermissionsBoundary: arn:aws:iam::...:policy/Boundary
aws s3api get-bucket-policy --bucket <bucket>	Resource policy (JSON with Allow/Deny statements)

Interpret the Evidence

- A deny in an SCP overrides any IAM allow.
- A permission boundary can restrict what the role can do.
- A session policy (e.g., from STS) can further limit permissions.
- A resource policy (e.g., S3, KMS, SQS) can deny access, even with an IAM allow.
- The final decision is the combined effect of all applicable policies.

Common Root Causes

- Explicit deny in an SCP (Organization level).
- Permission boundary is more restrictive.
- Session policy limits the allowed actions.
- Resource policy denies the principal or account.
- Condition keys (e.g., IP, VPC, MFA) not satisfied.

Fix and Verify

1. Remove or adjust the explicit deny (SCP, boundary, session, or resource policy).
2. Align all policies to allow the required action.
3. Check and satisfy any policy conditions (e.g., MFA, IP, VPC).
4. Re-test the action with the same role/credentials.
5. Use the policy simulator to confirm the final result is Allow.

Verification

aws iam simulate-principal-policy	EvalDecision: allowed
aws sts get-caller-identity	Correct role and account
aws s3 ls s3://<bucket>	Lists objects (access works)
aws cloudtrail lookup-events	No AccessDenied for the action

The Application Suddenly Cannot Decrypt Its Secrets

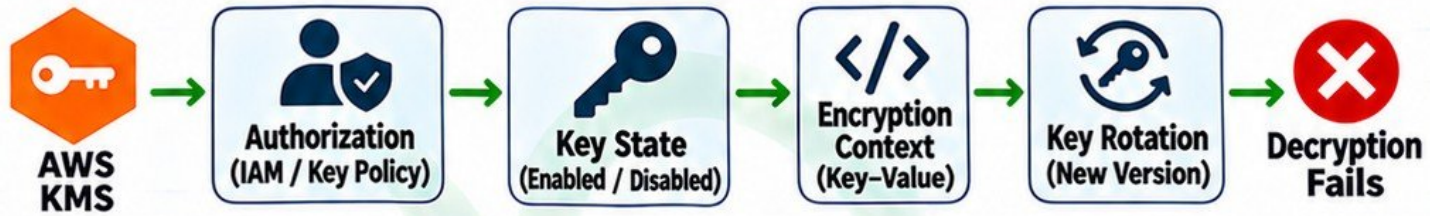
Diagnose KMS authorization, key state, encryption context, and rotation dependencies.

Scenario

The application suddenly fails to decrypt its secrets (e.g., database credentials or API keys) even though the secrets still exist.

Investigation Flow

1. Check KMS authorization and permissions.
2. Verify the key state and key policy.
3. Validate the encryption context (if used).
4. Review key rotation and new key versions.
5. Confirm the dependency (e.g., Secrets Manager, SSM).



Key Commands and Expected Output

Command	Expected Output (Example)
aws kms describe-key --key-id <key-id>	KeyState: Enabled KeyRotationEnabled: true
aws kms get-key-policy --key-id <key-id>	JSON policy showing Allow: kms:Decrypt for the role
aws kms decrypt \ --ciphertext-blob fileb://enc.bin \ --encryption-context env=prod	Plaintext (e.g., secret value) or AccessDeniedException
aws secretsmanager get-secret-value \ --secret-id app/db	SecretString (if KMS decryption succeeds)
aws kms list-aliases	Lists the alias and key ID (e.g., alias/app-secrets)

Interpret the Evidence

- AccessDenied means the caller lacks kms:Decrypt permission (IAM or key policy).
- The key must be in Enabled state to decrypt.
- A mismatched or missing encryption context (e.g., env=prod) will fail decryption.
- A new rotated key version may not be accessible by the application or policy.
- The issue can also come from a dependent service (e.g., Secrets Manager or SSM) that uses the KMS key.

Common Root Causes

- Missing or incorrect kms:Decrypt permission.
- Key is Disabled, PendingDeletion, or in the wrong region.
- Encryption context does not match the original.
- Newly rotated key not allowed by policy or app config.
- Dependency (Secrets Manager/SSM) misconfigured.

Fix and Verify

1. Grant or update the required KMS permissions (kms:Decrypt).
2. Ensure the key is Enabled and in the correct region.
3. Use the correct encryption context (key–value pairs).
4. Update the application to support the rotated key (if needed).
5. Re-test decryption and confirm the secret is returned.

Verification

aws kms describe-key --key-id <id>	KeyState: Enabled
aws kms decrypt --encryption-context env=prod	Returns plaintext (success)
aws secretsmanager get-secret-value --secret-id app/db	SecretString returned
aws kms get-key-policy --key-id <id>	Policy includes kms:Decrypt for the application role

The Certificate Is Valid, but TLS Connections Still Fail

Investigate SNI, certificate chains, protocol negotiation, and backend encryption.

Scenario

The TLS certificate is valid (not expired and correct domain), but TLS connections still fail (e.g., handshake error, certificate error, or timeout).

Investigation Flow

1. Verify the hostname and SNI configuration.
2. Check the certificate chain (intermediate and root).
3. Confirm TLS protocol and cipher compatibility.
4. Inspect backend (origin) encryption and settings.
5. Review logs and test the full TLS handshake.



Key Commands and Expected Output

Command	Expected Output (Example)
openssl s_client -connect example.com:443 -servername example.com	Verify return code: 0 (ok) and full certificate chain
curl -v https://example.com	TLS handshake succeeds (200 OK or expected response)
openssl s_client -connect example.com:443 -tls1_2	Tests TLS 1.2 (succeeds if supported)
openssl s_client -connect example.com:443 -tls1_3	Tests TLS 1.3 (succeeds if supported)
echo | openssl s_client -showcerts -connect example.com:443	Displays the full chain (leaf, intermediate, root)

Interpret the Evidence

- SNI mismatch can return the wrong certificate or cause handshake failure.
- An incomplete certificate chain (missing intermediate) can break trust.
- Protocol or cipher mismatch (e.g., TLS 1.0 / weak ciphers) may be rejected by the server.
- Backend encryption misconfiguration (e.g., TLS not enabled or wrong hostname) can fail the connection.
- Logs and handshake output reveal the exact TLS error (e.g., unknown_ca, handshake_failure).

Common Root Causes

- SNI not configured or hostname mismatch.
- Missing intermediate certificate in the chain.
- Unsupported TLS version or cipher suite.
- Backend (origin) not using the expected TLS settings.
- Certificate installed on the wrong host or listener.

Fix and Verify

1. Configure the correct SNI and hostname.
2. Install the complete certificate chain (leaf + intermediate + root).
3. Enable a supported TLS version and secure ciphers (e.g., TLS 1.2/1.3).
4. Fix backend TLS settings (e.g., enable HTTPS, correct certificate, or hostname).
5. Re-test the connection and confirm a successful handshake.

Verification

openssl s_client ... -servername <host>	Verify: return code: 0 (ok)
curl -v https://<host>	Returns 200 OK
openssl s_client ... -showcerts	Shows full, trusted chain
Check TLS protocol	TLS 1.2 or 1.3 negotiated
Review server/back-end logs	No TLS or handshake errors

The Cross-Account Service Worked Yesterday but Fails Today

Trace assumed roles, trust policies, expired credentials, and private connectivity.

Scenario

A cross-account service that accessed resources successfully yesterday now fails (e.g., AccessDenied, AssumeRole failed, or connection timeout).

Investigation Flow

1. Verify the AssumeRole call and current credentials.
2. Check the role's trust policy in the target account.
3. Confirm the permission policy for required actions.
4. Check for credential expiration (STS, tokens, keys).
5. Validate private connectivity (VPC, endpoint, peering).



Key Commands and Expected Output

Command	Expected Output (Example)
aws sts get-caller-identity	Account: 11112222333 Arn: arn:aws:sts::...:assumed-role/...
aws sts assume-role \ --role-arn arn:aws:iam::2222: role/CrossAccountRole \ --role-session-name test	Returns temporary credentials (AccessKeyId, SecretAccessKey, SessionToken)
aws iam get-role \ --role-name CrossAccountRole	Shows TrustPolicy and PermissionsPolicy
aws iam get-role-trust-policy --role-name CrossAccountRole	Trust relationship (Principal, Condition)
aws ec2 describe-vpc-endpoints --region us-east-1	Lists VPC endpoints (e.g., com.amazonaws.us-east-1.s3)

Interpret the Evidence

- AssumeRole failing usually means a trust policy or permission change.
- AccessDenied can come from updated IAM policies or conditions (e.g., IP, MFA).
- Expired STS credentials or tokens will cause authentication failures.
- Recent changes (role policy, SCP, or resource policy) can block access today.
- Private connectivity issues (VPC endpoint, peering, or route) can also cause the failure.

Common Root Causes

- Trust policy changed or no longer allows the caller.
- Permissions removed or narrowed (IAM policy or SCP).
- Expired or revoked credentials (STS, token, key).
- New condition (e.g., IP range, MFA, external ID) not met.
- Crivate connectivity broken (VPC endpoint, route, DNS).

Fix and Verify

1. Restore or update the trust policy to allow the caller (account, role, and conditions).
2. Ensure the role permission policy includes the required actions.
3. Refresh credentials (re-run AssumeRole or rotate keys).
4. Verify network connectivity (VPC endpoint, peering, DNS).
5. Re-test the cross-account access and confirm it works.

Verification

aws sts assume-role ...	Returns temporary credentials
aws sts get-caller-identity	Shows the assumed role (target account)
aws s3 ls (or target action)	Succeeds (e.g., bucket listing)
aws ec2 describe-vpc-endpoints	Endpoint is Available
Check CloudTrail (AssumeRole)	Successful AssumeRole event (today)

The Database CPU Is Normal, but Queries Take 30 Seconds

Investigate locks, connection pools, wait events, and storage latency.

Scenario

The database CPU is normal, but simple and known queries now take ~30 seconds to complete, causing slow application responses.

Investigation Flow

1. Check for blocking sessions and locks.
2. Review connection pool usage and saturation.
3. Analyze wait events to find the bottleneck.
4. Check storage I/O latency and throughput.
5. Correlate findings and confirm the root cause.



Key Commands and Expected Output

Command	Expected Output (Example)
SHOW PROCESSLIST; -- (MySQL)	Shows active queries and blocking session (Id, State, Time).
SELECT * FROM pg_locks; -- (PostgreSQL)	Lists locked objects and blocking PID.
SELECT * FROM pg_stat_activity;	Shows active sessions and wait_event (e.g.,I/O, lock).
SHOW STATUS LIKE 'Threads_connected';	Current and max connections (e.g., Threads_connected: 480).
EXPLAIN (ANALYZE) SELECT ...;	Query plan and actual time (e.g., Execution Time: 30.1s).

Interpret the Evidence

- Normal CPU with high query time usually indicates waiting (locks, I/O, or resources).
- Blocking locks can make queries wait for other transactions to finish.
- A saturated connection pool causes queuing and delays.
- Wait events (e.g., I/O, lock, buffer) show where time is spent.
- High storage latency (e.g., EBS/IOPS) can slow reads and writes, even with low CPU.

Common Root Causes

- Long-running transaction holding locks.
- Connection pool exhausted or misconfigured.
- High storage latency or low IOPS/throughput.
- Inefficient query (missing index or bad plan).
- Resource contention (e.g., I/O, memory, or FSM).

Fix and Verify

1. Identify and resolve blocking locks (commit, rollback, or kill the session).
2. Tune the connection pool (right size, timeouts, idle connections).
3. Optimize slow queries (add indexes, fix query plan).
4. Improve storage performance (increase IOPS/throughput or switch storage class).
5. Re-test the query and monitor wait events and latency.

Verification

Check	Expected Result
Slow query re-test	Completes in expected time (e.g., < 1s)
Locks check	No blocking sessions
Connection pool	Usage within safe limit (e.g., < 70–80%)
Wait events	Normal (no persistent lock or I/O waits)
Storage latency	Within baseline (e.g., < 5–10 ms)

The Message Queue Is Growing While Consumers Report Success

Diagnose retry loops, visibility timeouts, poison messages, and consumer throughput.

Scenario

The message queue keeps growing even though consumers report successful processing (e.g., "processed successfully" in logs or metrics).

Investigation Flow

1. Compare queue growth vs. real consumption rate.
2. Check for retry loops and redeliveries.
3. Validate the visibility timeout configuration.
4. Identify poison messages that never finish.
5. Assess consumer throughput and scaling limits.



Key Commands and Expected Output

Command (AWS SQS Example)	Expected Output (Example)
`aws sqs get-queue-attributes \ --queue-url <url> \ --attribute-names ApproximateNumberOfMessages`	`"ApproximateNumberOfMessages": "12500"` (growing backlog)
`aws sqs get-queue-attributes \ --queue-url <url> \ --attribute-names ApproximateNumberOfMessagesNotVisible`	`"ApproximateNumberOfMessagesNotVisible": "250"` (in processing)
`aws sqs get-queue-attributes \ --queue-url <url> \ --attribute-names VisibilityTimeout`	`"VisibilityTimeout": "30"` (seconds)
`aws sqs list-dead-letter-source-queues --queue-url <url>`	Returns DLQ mapping (if configured)

Interpret the Evidence

- Queue growing + high redeliveries indicates a retry loop.
- A visibility timeout that's too short causes the same message to be reprocessed.
- Poison messages repeatedly fail and return to the queue.
- Consumer "success" logs may be misleading (e.g., partial success, auto-retry, or not acknowledging correctly).
- Low consumer throughput or insufficient scaling can also cause backlog.

Common Root Causes

- Retry loop due to transient or logic errors.
- Visibility timeout shorter than processing time.
- Poison message (invalid data, schema, or format).
- Consumer not acknowledging (ACK) messages.
- Insufficient consumer throughput or scaling.

Fix and Verify

1. Break the retry loop (fix the root error or add proper retry with backoff).
2. Set an appropriate visibility timeout (e.g., > max processing time, e.g., 2–5 minutes).
3. Isolate poison messages (use a Dead-Letter Queue).
4. Increase consumer throughput (more consumers, higher concurrency, or autoscaling).
5. Monitor the queue after changes to confirm the backlog is decreasing.

Verification

Check	Expected Result
Queue size (ApproximateNumberOfMessages)	Decreases over time (e.g., 12,500 → 0)
Redrive / retry count	Stops increasing
Visibility timeout	Set ≥ processing time (e.g., 300s)
DLQ messages	Only failed/poison messages present
Consumer throughput (msgs/sec)	Increases and keeps up with incoming rate

The Deployment Succeeded, but Production Is Serving Two Application Versions

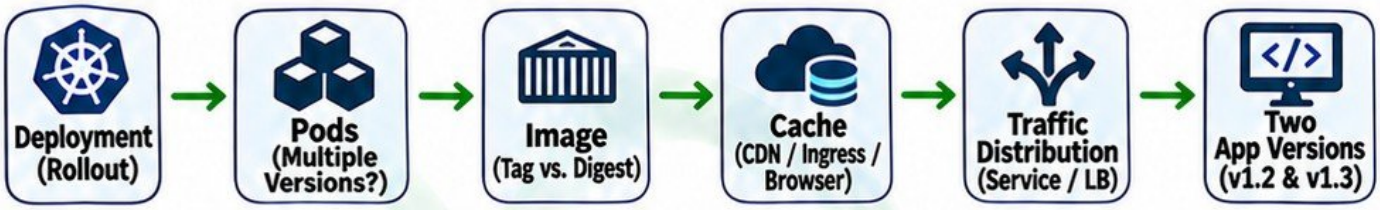
Investigate rollout state, image digests, caching, and traffic distribution.

Scenario

The deployment completed successfully, but users are seeing two different application versions in production (e.g., v1.2 and v1.3) at the same time.

Investigation Flow

1. Check the rollout and replica state.
2. Verify the container image and image digest.
3. Inspect caching (CDN, ingress, browser).
4. Analyze traffic distribution (service, ingress, LB).
5. Confirm the active version and clean up residuals.



Key Commands and Expected Output

Command	Expected Output (Example)
kubectl rollout status deployment/<app>	deployment "<app>" successfully rolled out
kubectl get pods -l app=<app> -o wide	Pods with mixed images (e.g., v1.2 and v1.3)
kubectl get deploy <app> -o =jsonpath='{.spec.template.spec. containers[0].image}'	myapp:v1.3 (or a digest: sha256:abc123...))
kubectl describe ingress <ing>	Shows backend services and routing rules
curl -I https://<app-domain>	Shows headers (e.g., cache, version)

Interpret the Evidence

- Mixed pod versions = rollout not fully complete or old replicas still running.
- Image tag may be reused; different digests can serve different builds.
- CDN/ingress/browser cache can return an older version.
- Traffic may be split across service endpoints, AZs, or target groups.
- Confirm the active image digest and ensure all traffic points to the new version.

Common Root Causes

- Rolling update not fully completed (old pods still alive).
- Same image tag (e.g., latest) pointing to different builds.
- Caching (CDN, ingress, or browser) serving old content.
- Traffic split across multiple backends or target groups.
- Stale replicas from a previous release or another service.

Fix and Verify

1. Complete or force the rollout to replace old pods (e.g., kubectl rollout restart).
2. Use immutable image tags or image digests (e.g., myapp:v1.3 or sha256:...).
3. Invalidate or reduce cache (CDN/ingress/browser).
4. Ensure traffic routes only to the new replica set.
5. Verify all pods serve the same version and monitor for consistency.

Verification

Check	Expected Result
kubectl get pods -l app=<app>	All pods run the same version (e.g., v1.3)
kubectl rollout status deploy/<app>	Rollout complete (success)
curl -I https://<app-domain>	Consistent headers (new version)
Check CDN/ingress cache	Cache cleared or updated
Monitor logs/metrics	Only one application version in use

Monitoring Shows Green While Customers Cannot Complete Transactions

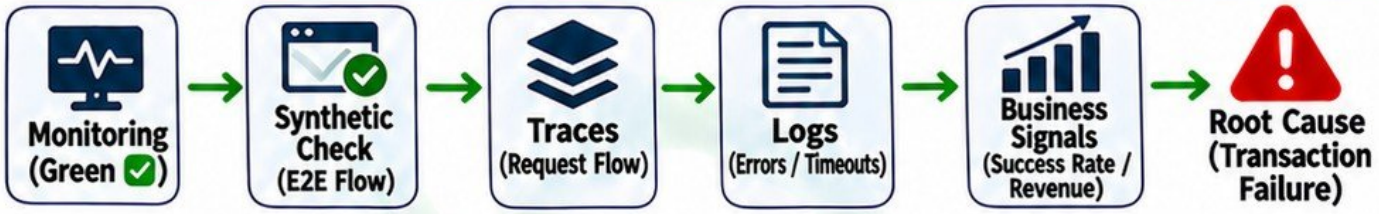
Build evidence from synthetic checks, traces, logs, and business-level signals.

Scenario

All system monitors are green (healthy), but customers cannot complete transactions (e.g., payments fail or orders do not finish).

Investigation Flow

1. Run synthetic (end-to-end) transaction checks.
2. Trace a failed transaction across services.
3. Analyze application and integration logs.
4. Check business-level signals (success rate, revenue).
5. Correlate findings and identify the real failure.



Key Commands and Expected Output

Command (Example)	Expected Output (Example)
curl -s -o /dev/null -w "%{http_code} %{time_total}\n" https://app.example.com/checkout	200 0.42 (synthetic checkout endpoint is reachable)
kubectl logs deploy/checkout --tail=100	Payment service timeout or error (e.g., 504 or connection error)
kubectl trace request <trace-id> (or use Jaeger/Tempo UI)	End-to-end trace showing the failing service (e.g., payment-gateway)
kubectl get svc,ep -payment-gateway	Service and endpoints are present and healthy (e.g., 2/2 endpoints)
grep -i "payment|transaction" app.log	Error details (e.g., upstream timeout, invalid response, or 5xx error)

Interpret the Evidence

- ✓ Green monitoring often checks infrastructure (e.g., CPU, memory, pod health) but may miss business flow failures.
- ✓ Synthetic checks validate the real customer journey (e.g., add to cart → pay → confirm).
- ✓ Traces show where the transaction fails and how long each step takes.
- ✓ Logs provide the exact error and context (e.g., timeout, 4xx/5xx, or dependency failure).
- ✓ Business signals (e.g., success rate, revenue drop, failed payments) confirm customer impact.

Common Root Causes

- Monitoring gap (infrastructure healthy, but business flow failing).
- Downstream dependency issues (e.g., payment, API).
- Timeouts, rate limits, or partial service outage.
- Logic or configuration error in the transaction flow.
- Lack of synthetic and business-level monitoring.

Fix and Verify

1. Add synthetic end-to-end checks for key transactions (e.g., checkout, payment).
2. Fix the failing dependency (e.g., payment API, timeout, or configuration).
3. Update monitoring to include business metrics (e.g., transaction success rate, revenue, failure rate).
4. Re-run synthetic checks and traces to confirm the fix.
5. Monitor for stability and set alerts on business-level signals.

Verification

Check	Expected Result
Synthetic checkout test	Completes successfully (e.g., 200, < 1s)
Trace for new transaction	All services succeed (no errors or timeouts)
Application logs	No recurring errors (e.g., payment/5xx)
Business metrics	Success rate returns to baseline (e.g., ≥ 99%)
Real customer test	Transaction completes end-to-end

The Disaster Recovery Failover Succeeded, but the Business Cannot Operate

Validate dependencies, data freshness, DNS, credentials, and recovery objectives.

Scenario

The disaster recovery (DR) failover completed successfully, but key business functions still do not work (e.g., outdated data, failed integrations, or login errors).

Investigation Flow

1. Verify all critical dependencies and integrations.
2. Check data freshness and replication status.
3. Validate DNS routing and propagation.
4. Confirm credentials, secrets, and access permissions.
5. Compare results against RTO and RPO targets.



Key Commands and Expected Output

Command (Example)	Expected Output (Example)
aws rds describe-db-clusters --region <dr>	Engine, status: "available" (in DR region)
aws s3 sync s3://primary/ s3://dr-bucket --dryrun	Shows objects to sync (or 0 if in sync)
dig +short app.example.com	Returns DR IP (e.g., 52.x.x.x)
aws ssm get-parameter \ --name /app/db-url	Returns DR database URL (e.g., postgres://dr-db:5432)
aws cloudwatch get-metric-statistics \ --metric-name ReplicationLag	ReplicationLag: 0–few seconds (healthy)

Interpret the Evidence

- Failover can succeed technically but still miss critical business requirements.
- Data may not be current (RPO not met).
- DNS may still point to the primary site (or not fully propagated).
- Applications may use outdated or missing credentials/secrets in the DR environment.
- Dependent services (payments, email, APIs) may not be available or reconfigured in DR.

Common Root Causes

- Data not fully replicated (RPO breach).
- DNS not updated or still caching the old IP.
- Missing or invalid credentials/secrets.
- Unconfigured or unavailable dependent services.
- DR environment not fully validated against RTO/RPO.

Fix and Verify

1. Sync or restore data to meet the RPO target.
2. Update and verify DNS (reduce TTL and confirm resolution to DR).
3. Reconfigure and validate all credentials/secrets.
4. Bring up and test all dependent services and integrations (e.g., payment, email, queues).
5. Run end-to-end business checks and confirm RTO/RPO are met.

Verification

Check	Expected Result
Data freshness	Within RPO (e.g., ≤ 5 minutes)
DNS resolution	Points to DR (correct IP/endpoint)
Credentials / access	Valid and working (no auth errors)
Dependencies	All critical integrations are healthy
Business functionality	Key user flows work end-to-end
Recovery objective	RTO and RPO achieved (e.g., RTO ≤ 30 min)

Final Senior SRE Challenge
A Multi-Cloud Incident Investigation

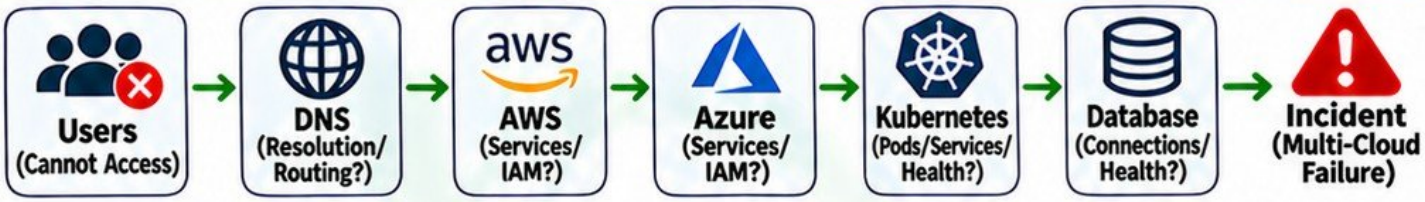
Investigate a multi-cloud (AWS, Azure, Kubernetes, DNS, IAM, and database) failure and produce a defensible incident report.

Scenario

A production outage affects a business running across AWS, Azure, and Kubernetes. Users cannot access the application, and database operations are failing.

Investigation Flow

1. Triage: confirm scope, impact, and symptoms.
2. Check DNS, multi-cloud connectivity, and routing.
3. Validate IAM permissions and access paths.
4. Inspect Kubernetes workloads and dependencies.
5. Diagnose database health and cross-cloud integrations.



Key Commands and Expected Output

Command (Example)	Expected Output (Example)
dig +short app.example.com	IP address (e.g., 52.x.x.x)
aws sts get-caller-identity	AWS account and IAM user/role
az account show --output table	Azure subscription and tenant
kubectl get pods -n prod	Shows pod status (e.g., Running/CrashLoopBackOff)
kubectl describe pod <pod> -n prod	Events and error details (e.g., image pull, permission, DNS)
kubectl logs <pod> -n prod	Application errors (e.g., DB timeout)
psql -h <db-endpoint> -U <user> -c "\conninfo"	Connected to database (or error)

Interpret the Evidence

- DNS: is it resolving to the correct and healthy endpoints?
- IAM: are roles, policies, or federated access blocking the services?
- Kubernetes: are pods healthy, scaled, and able to reach dependencies?
- Database: is it available, accepting connections, and performing as expected?
- Multi-cloud: are network routes, peering, and service dependencies working end-to-end?

Common Root Causes

- DNS misconfiguration or stale records.
- IAM permission changes or expired credentials.
- Kubernetes deployment or dependency failure (e.g., image, config, secret).
- Database outage, connection limit, or lock contention.
- Cross-cloud network or routing misconfiguration.

Fix and Recover

1. Restore DNS to the correct, healthy endpoints.
2. Fix IAM access (update policy, role, or credentials).
3. Stabilize Kubernetes (restart, roll back, or scale).
4. Restore database (unlock, scale, or failover).
5. Verify full multi-cloud service health and resume normal traffic.

Incident Report (Defensible)

Section	What to Include
Incident Summary	What happened, impact, duration, severity.
Timeline	Key events from detection to full recovery.
Root Cause	The primary cause with evidence.
Resolution	Actions taken and how service was restored.
Impact Analysis	Affected users, services, and business impact.
Lessons & Prevention	Improvements (monitoring, IAM, DNS, K8s, DB, DR).